



Orvio — Technical Architecture (architecture.md)

Version: 0.1 (MVP-ready)

Last updated: 2026-09-28

Owner: Engineering (Code X)

Audience: Frontend, Backend, DevOps, QA, AI coding agents

0. How to Use This Document

This is the **technical source of truth** for Orvio. It translates the PRD into a buildable system. Use it to:

- Understand **boundaries** (what lives where and why).
- Implement features without inventing parallel architectures.
- Make consistent decisions about **security, tenancy, and data**.
- Onboard new developers and AI coding agents quickly.

Rule: If something is not documented here, do not introduce it without an ADR (Architecture Decision Record).

1. Architectural Overview

1.1 System Summary

Orvio is a **multi-tenant SaaS** for Nigerian SMBs that unifies **inventory, POS, and WhatsApp orders with local payments**. The MVP is a **modular monolith** with clear domain boundaries, designed to evolve into services only when needed.

1.2 Major Components

- **Web App (React + TypeScript):** Admin/merchant UI for inventory, POS, orders, reports, settings.
- **Backend API (Node.js + Fastify + TypeScript):** REST API, auth, business logic, integrations.
- **Database (PostgreSQL):** Relational data, transactions, audit logs, multi-tenant isolation.
- **Background Workers (Node.js):** Email (later), webhooks processing, scheduled jobs, imports/exports.
- **External Integrations:** WhatsApp Business API (via BSP), Payment gateways (Paystack primary, Flutterwave/Moniepoint secondary).

- **Object Storage (S3-compatible):** Receipts/assets (future), product images (if stored).
- **Observability:** Structured logs, metrics, error tracking (e.g., Sentry), request IDs.

1.3 High-Level Diagram

```

flowchart TD
    subgraph Client
        Web[React Web App]
        POS[POS Client (Web)]
    end

    subgraph Edge
        CDN[CDN / Static Hosting]
        APIGW[API Gateway / Load Balancer]
    end

    subgraph Application
        API[Fastify API (Modular Monolith)]
        Worker[Background Workers]
    end

    subgraph Data
        DB[(PostgreSQL)]
        Store[(Object Storage)]
    end

    subgraph External
        WA[WhatsApp BSP API]
        PAY[Payment Gateways<br/>Paystack/Flutterwave/Moniepoint]
    end

    Web --> CDN
    POS --> CDN
    CDN --> APIGW
    APIGW --> API
    API --> DB
    API --> Store
    API --> WA
    API --> PAY
    API --> Worker
    Worker --> DB
    Worker --> WA
    Worker --> PAY
  
```

1.4 Data & Request Flow (Typical)

1. User opens app → CDN serves static assets.
2. Frontend calls API (/api/v1/...) with JWT in Authorization header.
3. API validates auth, resolves org context, enforces RBAC, executes business logic.
4. API reads/writes DB within transactions; logs audit events.

5. For payments/WhatsApp, API calls external services; webhooks arrive at dedicated endpoints.
6. Workers process async tasks (webhook verification, retries, scheduled jobs).

1.5 Authentication & Authorization Flow (Summary)

- **AuthN:** Email/phone + password → JWT access + refresh tokens (rotating).
- **AuthZ:** Every request includes `org_id` (from token or body); server enforces role/permission checks per resource.
- **Frontend:** Uses tokens for UX guards; **backend is the security boundary.**

2. Architectural Decision Record (Summary)

Decision	Options	Recommended	Reason	Trade-offs
Frontend framework	React + Vite, Next.js	React + Vite + TS	MVP is app-like, SEO not critical; simpler than SSR/SSG.	No SSR benefits; easier to host static.
Backend framework	Fastify, Express, NestJS	Fastify + TS	Fits existing skills; lightweight, plugin ecosystem; no need for NestJS complexity.	Less opinionated than Nest; requires discipline.
API style	REST, GraphQL, gRPC	REST (JSON)	Aligns with PRD, simple for AI agents, easy caching.	Less flexible than GraphQL for ad-hoc queries.
Database	PostgreSQL, MySQL, Mongo	PostgreSQL	Relational data, transactions, reporting, constraints, multi-tenant isolation.	Operational complexity vs serverless DBs.
ORM / Query	Prisma, Kysely, raw SQL	Prisma (MVP)	Type safety, migrations, DX; good for AI agents.	Some runtime overhead; vendor lock-in.
Auth	Sessions, JWT, OAuth	JWT (access+refresh)	Stateless API, mobile-friendly later; simple with Fastify.	Token revocation needs strategy.
State mgmt	Redux, Zustand, Context	Zustand + TanStack Query	Simple global state; server state via Query.	Extra deps but minimal.
Caching	None, Redis, in-memory	In-memory + HTTP cache (MVP)	MVP scale doesn't need Redis yet.	Limited distributed cache.
Queue	None, BullMQ, SQS	BullMQ + Redis (Phase 2)	MVP can process sync; defer queue.	Add infra later.
Hosting	VPS, PaaS, Serverless	PaaS (Render/Railway/Fly) (MVP)	Fast setup, managed DB, CI/CD friendly.	Less control, cost at scale.
Observability	Custom, Sentry + logs	Sentry + structured logs	Fast error visibility; logs for audit.	Cost, vendor.

3. Technology Stack

3.1 Frontend

- **React 18 + TypeScript**
- **Vite** (build/dev)
- **React Router v6+** (routing)
- **TanStack Query** (server state, caching, background refresh)
- **Zustand** (light global UI/auth state)
- **React Hook Form + Zod** (forms + validation)
- **Tailwind CSS or Chakra UI** (styling; choose one)
- **Axios or fetch wrapper** (API client)

Why not Next.js? Orvio MVP is a logged-in app; SEO/SSR not required. Vite + SPA is simpler to deploy and reason about.

3.2 Backend

- **Node.js 20+ LTS**
- **Fastify 4+ with TypeScript**
- **Prisma ORM**
- **PostgreSQL 15+**
- **JWT** (access + refresh), **bcrypt/argon2**
- **Zod or @sinclair/typebox** (request validation)
- **Pino** (structured logging)
- **node-cron or BullMQ** (later) for scheduled jobs
- **dotenv + secret manager** (env vars in PaaS)

3.3 DevOps & Tooling

- **Git + GitHub/GitLab**
- **CI:** GitHub Actions (lint, typecheck, test, build, deploy)
- **CD:** PaaS pipelines (staging → production)
- **Docker** (optional for local; PaaS handles prod)
- **Sentry** (error tracking)
- **Log provider** (PaaS logs or hosted ELK later)

4. Frontend Architecture

4.1 Application Structure

```
web/
├── public/
├── src/
│   ├── app/                # App root, providers, router
│   ├── features/           # Feature modules (inventory, pos, orders, payments, wa, i
│   ├── components/         # Shared UI components
│   ├── layouts/            # App shells (dashboard, auth)
│   ├── hooks/              # Shared hooks
│   ├── lib/                # Utilities (api client, auth helpers, constants)
│   ├── services/           # API service wrappers per domain
│   ├── stores/             # Zustand stores (auth, ui)
│   ├── types/              # Shared TS types
│   └── utils/              # Helpers (formatters, validators)
├── index.html
├── vite.config.ts
└── tsconfig.json
```

Why this structure?

- `features/` enforces **domain boundaries** (inventory vs pos vs orders).
- `services/` centralizes API calls; components don't call endpoints directly.
- `stores/` holds minimal global state (auth, UI flags); server data lives in TanStack Query.

4.2 Routing

- Public routes: `/login`, `/register`, `/reset-password`
- Protected routes: `/dashboard`, `/inventory`, `/pos`, `/orders`, `/reports`, `/settings`
- Route guards check **auth token** and **role permissions** (UX only; backend still enforces).

4.3 State Management

- **Server state:** TanStack Query (queries/mutations). Cache keys include `org_id`.
- **Global UI/auth:** Zustand (current org, user, sidebar state).
- **Local component state:** React `useState/useReducer`.
- **URL state:** Filters, pagination, search in query params.

4.4 Loading, Error, Empty States

- **Skeleton:** Dashboards, lists (products, orders).
- **Spinners:** Buttons, small actions.
- **Empty states:** Clear CTAs (e.g., "Add your first product").
- **Error states:** User-friendly messages + retry; log correlation ID.

5. Frontend Feature Architecture (Examples)

5.1 Inventory Feature (features/inventory/)

- **Components:** ProductList, ProductForm, StockTable, AdjustmentModal.
- **Hooks:** useProducts, useCreateProduct, useAdjustStock.
- **API:** services/inventory.ts (CRUD products, stock adjustments).
- **Permissions:** Hide “Adjust” button if role lacks permission (backend still checks).
- **Validation:** Zod schemas for forms; server validates again.

5.2 POS Feature (features/pos/)

- **Components:** Cart, ItemScanner, PaymentModal, ReceiptPreview.
- **Offline:** Local queue (IndexedDB or localStorage) for sales when offline; sync when online.
- **Optimistic updates:** Add to cart instantly; rollback on sync failure.

5.3 WhatsApp Orders (features/orders/whatsapp/)

- **Components:** WhatsAppSettings, OrderList, OrderDetail.
- **API:** services/whatsapp.ts (enable channel, fetch orders).
- **Notifications:** In-app toasts for new orders.

6. Backend Architecture

6.1 Modular Monolith Structure

```
api/
├── src/
│   ├── app.ts           # Fastify app factory
│   ├── server.ts        # Entry point
│   ├── config/          # Env, constants
│   ├── modules/         # Domain modules
│   │   ├── auth/
│   │   ├── org/
│   │   ├── inventory/
│   │   ├── pos/
│   │   ├── orders/
│   │   ├── payments/
│   │   └── whatsapp/
│   ├── plugins/         # Fastify plugins (auth, cors, logging)
│   ├── middleware/      # Auth, org context, error handler
│   ├── lib/            # Shared utils (jwt, hash, logger)
│   ├── infrastructure/  # DB, external clients (Paystack, WhatsApp)
│   └── shared/          # DTOs, schemas, types
└── prisma/
```

```
| └─ schema.prisma
└─ tests/
```

6.2 Layer Responsibilities

- **Routes:** HTTP concerns (method, path, validation).
- **Controllers/Handlers:** Parse request, call service, return response.
- **Services:** Business logic (stock calculations, order state transitions).
- **Repositories (Prisma):** Data access, transactions.
- **Domain:** Pure business rules (e.g., “cannot go negative stock”).

Rule: Business logic lives in **services/domain**, not routes.

7. API Architecture

7.1 Style

- **REST**, JSON, versioned under `/api/v1`.
- **Stateless**, JWT auth.
- **Consistent response shape:**

```
{
  "data": { ... },
  "meta": { "requestId": "..." }
}
```

- **Error shape:**

```
{
  "error": {
    "code": "RESOURCE_NOT_FOUND",
    "message": "The requested resource was not found.",
    "requestId": "..."
  }
}
```

7.2 Representative Endpoints

- `POST /api/v1/auth/register`
- `POST /api/v1/auth/login`
- `GET /api/v1/orgs/:id`
- `GET /api/v1/products?branchId=&search=&page=`
- `POST /api/v1/products`
- `POST /api/v1/stock/adjustments`

- POST /api/v1/pos/sales
- POST /api/v1/whatsapp/orders
- POST /api/v1/payments/links
- POST /api/v1/webhooks/paystack

7.3 Cross-Cutting Concerns

- **Validation:** Zod schemas per route.
- **Pagination:** page, pageSize, total, hasMore.
- **Filtering/Sorting:** Query params; whitelist fields.
- **Rate limiting:** Per org/user (Fastify plugin).
- **Idempotency:** Idempotency-Key header for POST sales/payments.

8. Database Architecture

8.1 Choice: PostgreSQL

- Relational data with **transactions** (critical for stock & payments).
- Strong **constraints** (unique SKU per org, FKs).
- Reporting & aggregations (sales, stock valuation).
- Row-level security possible later.

8.2 Core Entities (Conceptual ERD)

```
erDiagram
    ORGANIZATION ||--o{ BRANCH : has
    ORGANIZATION ||--o{ USER_MEMBERSHIP : includes
    ORGANIZATION ||--o{ PRODUCT : owns
    BRANCH ||--o{ STOCK_ITEM : holds
    PRODUCT ||--o{ STOCK_ITEM : tracked_in
    PRODUCT ||--o{ STOCK_MOVEMENT : generates
    BRANCH ||--o{ STOCK_MOVEMENT : occurs_at
    BRANCH ||--o{ SALES_ORDER : receives
    SALES_ORDER ||--o{ SALES_ORDER_ITEM : contains
    PRODUCT ||--o{ SALES_ORDER_ITEM : sold_as
    SALES_ORDER ||--o{ PAYMENT_INTENT : paid_by
    ORGANIZATION ||--o{ WHATSAPP_CHANNEL : configures
    WHATSAPP_CHANNEL ||--o{ WHATSAPP_ORDER : receives
    ORGANIZATION ||--o{ SUBSCRIPTION : billed_as
```

8.3 Key Constraints & Indexes

- `UNIQUE (org_id, sku) on Product.`
- `UNIQUE (org_id, phone_number) on WhatsAppChannel.`
- `Indexes: product.org_id, stock_item.branch_id, sales_order.org_id, payment_intent.order_id.`
- **Soft delete:** `deleted_at` on critical tables; queries filter `deleted_at IS NULL`.

9. Data Access Architecture

- **Prisma ORM** with **repository pattern** per module.
- **Transactions:** Wrap multi-step operations (sale → stock movement).
- **Connection pooling:** Managed by Prisma/DB driver.
- **Migrations:** Prisma migrate; never manual schema changes in prod.
- **Validation:** Zod schemas in API; Prisma types for DB.

10. Authentication Architecture

10.1 Flow (Login)

```
sequenceDiagram
    participant U as User
    participant F as Frontend
    participant A as API
    participant DB as Database

    U->>F: Enter credentials
    F->>A: POST /auth/login
    A->>DB: Verify user & hash
    DB-->>A: User record
    A->>A: Generate access + refresh tokens
    A-->>F: Tokens (httpOnly cookie or body)
    F->>U: Redirect to dashboard
```

10.2 Token Strategy

- **Access token:** Short-lived (15 min), in memory.
- **Refresh token:** Longer (7–30 days), `httpOnly` cookie or secure storage.
- **Rotation:** Refresh on use; revoke on suspicious activity.

10.3 Passwords

- **Hash:** bcrypt/argon2 with cost factor.
- **Reset:** Token-based, expiry 1 hour, one-time use.

11. Authorization / RBAC

- **Roles:** Owner, Manager, Cashier, Storekeeper, Accountant, Internal Admin.
- **Permissions:** Evaluated server-side per request (org + branch scope).
- **Enforcement:** Middleware checks role/permission before service call.
- **Frontend:** Hides UI; **backend is authoritative.**

12. Multi-Tenancy

- **Tenant = Organization.** Every row scoped by `org_id`.
- **Request context:** Extract `org_id` from token or validated body.
- **Isolation:** SQL `WHERE org_id = ?` on every query; repository enforces.
- **Threat model:** Prevent cross-org access via:
 - Mandatory `org_id` in context.
 - Repository base class injecting `org_id`.
 - Tests for tenant isolation.

13. State Management

- **Server state:** TanStack Query (cached, background refresh).
- **Client state:** Zustand (auth user, current org, UI flags).
- **URL state:** Filters, pagination, search.
- **Persistent:** Non-sensitive prefs in `localStorage`.

14. Caching Strategy

- **Browser:** HTTP cache for static assets (CDN).
- **Client:** TanStack Query cache (per org).
- **Server:** In-memory cache for config; no Redis in MVP.
- **Invalidation:** On mutation (create/update/delete).

15. Real-Time Architecture

- **MVP:** Polling for new orders/notifications (every 30–60s).
- **Future:** WebSockets/SSE for live POS dashboards.
- **Webhooks:** External (payments, WhatsApp) → API endpoints.

16. Background Jobs

- **MVP:** Synchronous where possible; simple cron for cleanup.
- **Phase 2:** BullMQ + Redis for:
 - Webhook retries.
 - Email notifications.
 - Imports/exports.
- **Idempotency:** Job handlers check processed flags.

17. File Storage

- **Provider:** S3-compatible (Cloudflare R2, AWS S3).
- **Upload:** Direct signed URL from API; frontend uploads directly.
- **Access control:** Signed URLs, expiry.
- **Validation:** MIME, size limits, virus scan (future).

18. Error Handling

- **Centralized Fastify error handler.**
- **Error codes:** AUTH_INVALID, FORBIDDEN, NOT_FOUND, CONFLICT, RATE_LIMITED, EXTERNAL_FAILURE.
- **Logging:** Structured (Pino) with `requestId`, `userId`, `orgId`.
- **User messages:** Generic; never leak stack traces.

19. Frontend Loading Architecture

- **Initial load:** Skeleton for dashboard/lists.
- **Route changes:** Show skeleton for main content.
- **Mutations:** Button spinners.
- **Background refresh:** TanStack Query stale-while-revalidate.

20. Observability

- **Logging:** Pino JSON logs; include `requestId`, `orgId`, `userId`.
- **Metrics:** Request latency, error rates, job durations.
- **Tracing:** Correlation IDs across services.
- **Error tracking:** Sentry (frontend + backend).

Never log: Passwords, tokens, secrets.

21. Security Architecture

- **OWASP Top 10:** Input validation, output encoding, parameterized queries.
- **Auth:** Rate limit login, brute-force protection.
- **Session:** Token rotation, revocation list (future).
- **Tenant isolation:** Enforced in repository layer.
- **Secrets:** Env vars in PaaS; never in code.
- **Webhooks:** HMAC signature verification (Paystack/WhatsApp). ^[1] ^[2]
- **Headers:** HSTS, CSP, X-Frame-Options, etc.

22. Performance Architecture

- **Frontend:** Code splitting per route, lazy load heavy components.
- **API:** Indexes on frequent filters, pagination, avoid N+1 queries.
- **Caching:** HTTP cache for static, Query cache for API.
- **Bottlenecks:** Stock queries, sales reports; optimize with indexes/materialized views later.

23. Scalability Architecture

- **Stage 1 (MVP):** Single API + DB (PaaS).
- **Stage 2 (Growth):** Add Redis, workers, CDN.
- **Stage 3 (Scale):** Read replicas, service separation (auth, payments, inventory).

Evolution: Modular monolith → extract services by domain when load/complexity demands.

24. Deployment Architecture

```
flowchart LR
    subgraph Dev
        Local[Local Dev]
    end
    subgraph Staging
        WebStg[Web Staging]
    end
```

```

    ApiStg[API Staging]
    DbStg[DB Staging]
end
subgraph Prod
    WebProd[Web Prod]
    ApiProd[API Prod]
    DbProd[DB Prod]
end
Local --> WebStg
Local --> ApiStg
WebStg --> WebProd
ApiStg --> ApiProd

```

- **Frontend:** Hosted on PaaS static or Netlify/Vercel.
- **Backend:** PaaS (Render/Railway/Fly).
- **DB:** Managed PostgreSQL (same PaaS or separate).
- **CDN:** For static assets.
- **SSL/TLS:** Managed by host.

25. Environment Strategy

Env	Purpose	DB	API URL	Notes
Local	Dev	Local Postgres	http://localhost:3000	Seed data
Dev	Integration	Shared dev DB	https://dev-api.orvio...	Bleeding edge
Staging	Pre-prod	Staging DB	https://staging-api.orvio...	Mirrors prod
Prod	Live	Prod DB	https://api.orvio...	Strict access

Secrets: Per env, never committed.

26. CI/CD

```

flowchart LR
    PR[Pull Request] --> Lint[Lint]
    Lint --> Type[Type Check]
    Type --> Test[Test]
    Test --> Build[Build]
    Build --> DeployStg[Deploy Staging]
    DeployStg --> E2E[E2E]
    E2E --> DeployProd[Deploy Prod]

```

- **CI:** GitHub Actions.
- **CD:** PaaS auto-deploy on main branch.

27. Testing Architecture

- **Frontend:** Vitest + React Testing Library (components, hooks).
- **Backend:** Vitest/Jest (unit, service, API integration).
- **E2E:** Playwright/Cypress (critical journeys).
- **Security:** AuthZ tests, tenant isolation tests.
- **DB:** Migration tests, constraint tests.

28. Project / Repository Structure

Monorepo (recommended):

```
orvio/  
├─ apps/  
│   ├── web/  
│   └─ api/  
├─ packages/  
│   ├── shared/  
│   └─ config/  
├─ docs/  
└─ infra/
```

- apps/web: React app.
- apps/api: Fastify API.
- packages/shared: Types, schemas.
- packages/config: ESLint, TS configs.

29. Code Ownership and Boundaries

Dependency direction:

```
UI → Features → API Client → HTTP API → Services → Domain → Repositories → DB
```

Prohibited:

- UI → DB directly.
- Services → UI.
- Routes → DB without service.

30. API / Frontend Contract

- **Shared types:** `packages/shared` (TypeScript).
- **OpenAPI:** Generate from Fastify schemas (future).
- **Validation:** Zod on both sides; runtime checks.

31. Database Migration Strategy

- **Tool:** Prisma migrate.
- **Process:** Dev → Staging → Prod.
- **Rollback:** Prisma `db execute` with down migrations.
- **Data migrations:** Separate scripts, idempotent.

32. Disaster Recovery

- **Backups:** Daily automated (PaaS).
- **Retention:** 7–30 days (configurable).
- **RPO:** < 24 hours.
- **RTO:** < 4 hours.
- **Restore tests:** Quarterly.

33. Third-Party Dependencies

Dependency	Purpose	Failure Impact	Mitigation
Paystack	Payments	Can't collect payments	Queue, retry, manual link fallback
WhatsApp BSP	Orders	No WhatsApp orders	Fallback to manual order entry
Sentry	Errors	Less visibility	Logs still available
PaaS	Hosting	Downtime	Multi-region later

34. Architectural Risks

Risk	Cause	Impact	Mitigation	Revisit
Token theft	Weak storage	Account takeover	httpOnly cookies, rotation	Post-MVP
Cross-org leak	Missing <code>org_id</code> filter	Data breach	Repository enforcement, tests	Now
Offline sync bugs	Conflict logic	Stock errors	Audit, manual adjustment UI	MVP
Vendor lock-in	PaaS/Prisma	Migration cost	Abstraction layers	Phase 2

35. Architecture Decision Records (ADRs)

ADR-001: Backend Framework

Status: Accepted

Context: Need Node.js backend; team knows Fastify.

Decision: Fastify + TypeScript.

Alternatives: Express (less structured), NestJS (more complex).

Reason: Fastify balances performance and simplicity; no need for NestJS overhead.

Trade-offs: Less opinionated; requires discipline.

Consequences: Define clear layer boundaries in code.

ADR-002: Database

Status: Accepted

Context: Relational data, transactions, reporting.

Decision: PostgreSQL + Prisma.

Alternatives: MySQL, Mongo.

Reason: Strong constraints, transactions, reporting.

Trade-offs: Operational complexity vs serverless DBs.

ADR-003: Frontend Framework

Status: Accepted

Context: Logged-in app, SEO not critical.

Decision: React + Vite + TS.

Alternatives: Next.js.

Reason: Simpler for MVP; no SSR needed.

Trade-offs: No SSR benefits.

36. Implementation Order

1. Repo setup (monorepo, configs).
2. Dev environment (local DB, seed).
3. Auth module (register, login, tokens).
4. Org & user membership.
5. Inventory module (products, stock).
6. POS module (sales, offline queue).
7. Orders & WhatsApp (catalog, orders, payment links).
8. Payments integration (Paystack webhooks).
9. Reports (basic sales/stock).
10. Observability (Sentry, logs).
11. CI/CD (staging, prod).

12. Testing (unit, integration, E2E).

37. AI Coding Agent Guidelines

Before changing code:

- Inspect existing module boundaries (`modules/*`).
- Search for existing services/repositories.
- Reuse `lib/` utilities (`jwt`, `hash`, `logger`).
- Do not bypass `authZ` or `org` context.

When implementing:

- Follow layer rules (`routes` → `services` → `repos`).
- Do not add new libs without ADR.
- Keep business logic out of UI.
- Preserve API contracts; update shared types.

Before finishing:

- Run lint, typecheck, tests.
- Verify affected workflows (`auth`, `org`, `inventory`).
- Check security (`org` isolation, permission checks).

38. Architectural Constraints

- React + TypeScript frontend.
- Node.js + Fastify + TypeScript backend.
- Business logic in `services/domain`, not `UI/routes`.
- Authorization enforced server-side.
- No direct DB access from frontend.
- Secrets never exposed to client.
- Tenant data isolated by `org_id`.
- New infra requires documented reason.

39. What Not to Build Yet

- Microservices.
- Kubernetes.
- Redis (unless queue needed).
- Elasticsearch.

- Complex event buses.
- Multiple databases.
- Dedicated data warehouse.

Introduce when: Clear scaling/complexity need (e.g., >10k daily active users, heavy search).

40. Final Architecture Summary

- **Stack:** React + Vite + TS | Node.js + Fastify + TS | PostgreSQL + Prisma.
- **System:** Modular monolith, multi-tenant, JWT auth, RBAC.
- **Frontend:** Feature-oriented, TanStack Query + Zustand.
- **Backend:** Layered (routes → services → repos), Prisma for data.
- **Security:** Org isolation, HMAC webhooks, OWASP mitigations. ^[2] ^[1]
- **Deployment:** PaaS (staging → prod), CI/CD via GitHub Actions.
- **Observability:** Structured logs, Sentry, correlation IDs.
- **Scalability:** Start simple; add Redis/workers/services as load grows.
- **Major risks:** Token security, cross-org leaks, offline sync.
- **Future:** Queues, advanced reports, eCommerce, logistics.

End of architecture.md



1. <https://www.kobosync.com/blog/2026-06-25-webhook-security-best-practices-nigeria>

2. <https://www.mctaba.com/learn/paystack/paystack-in-nigeria-complete-developer-guide>